

Algoritmi-Modulo algoritmi per bioinformatica

Hashing e Hash Table-Capitolo 11 CLRS

Roberto Posenato

versione 2026-1, 21/03/2026

Sommario

- 1 Introduzione
- 2 Tavole a indirizzamento diretto
- 3 Tavole hash
 - Funzione hash: come scegliere?
 - Funzione Hash: conversione verso gli interi
 - Hashing statico
 - Metodo divisione
 - Metodo moltiplicazione
 - Metodo moltiplica e scorri
 - Hashing randomizzato
- 4 Collisioni
 - Risoluzione mediante concatenamento
 - Risoluzione mediante indirizzamento aperto
 - Ispezione lineare
 - Doppio hashing
- 5 Esercizi

*“Tell me and I forget, teach me and I remember,
involve me and I learn.”*

Benjamin Franklin

“A student who achieves knowledge through free investigation and spontaneous effort will be able to retain that knowledge and will have acquired a methodology that can serve for a lifetime.”

Jean Piaget

Scopo della lezione

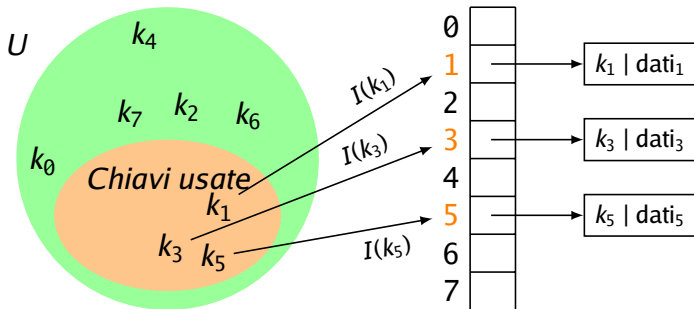
- In molte applicazioni, le operazioni di dizionario richieste sono solo INSERT, SEARCH e DELETE.

Esempi

- Il dizionario classico! La chiave è il lemma e l'informazione è la spiegazione dello stesso.
- L'elenco telefonico. La chiave è il nome dell'utente. L'informazione è il numero di telefono.
- Una *tabella hash* è una struttura dati efficace per implementare dizionari che richiedano solo le operazioni INSERT, SEARCH e DELETE.
- Anche se teoricamente l'operazione SEARCH può richiedere tempo $O(n)$ nel caso peggiore, si dimostra che nella pratica, sotto ipotesi ragionevoli, il tempo medio è $O(1)$.
- Una tabella hash è la generalizzazione del concetto di array: in un array, gli elementi sono individuati da un indice intero, in una tabella hash, gli elementi sono individuati dalla *chiave* dell'elemento.

Tavole a indirizzamento diretto

- Sia U l'insieme dei possibili valori delle chiavi degli elementi che si vogliono memorizzare in un dizionario.
- **Assunzioni fondamentali per l'indirizzamento diretto:**
 - $|U|$ non è troppo grande.
 - Che nessun elemento abbia chiave uguale a un altro elemento.
- In questo caso, è possibile convertire la chiave in un intero e usare un array T di lunghezza $|U|$ per memorizzare i riferimenti agli elementi che si vogliono memorizzare.



Esempio 1

- La chiave è una stringa di 2 caratteri nell'intervallo $[A, \dots, Z]$
- In Unicode, $[A, \dots, Z]$ è formato da 26 caratteri
- $|U| = 26 \cdot 26 = 676$
- Una conversione univoca di una chiave k verso gli interi è la funzione

$$I(k) = \sum_{i=0}^1 (k[i] - 'A') \cdot 26^i$$

- Esempio: “AZ” $\mapsto 0 \cdot 26^0 + 25 \cdot 26^1 = 650$
“ZZ” $\mapsto 25 \cdot 26^0 + 25 \cdot 26^1 = 675$
- Si deve allocare un array di Object di lunghezza $m = 676$ anche se si devono memorizzare un numero n di elementi con $n \ll m$.

Tavole hash

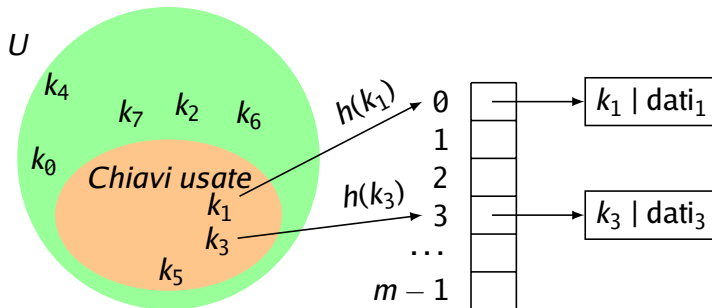
- Se $|U|$ è troppo grande, la tabella a indirizzamento diretto non è realizzabile.
- Se n (**numero di chiavi realmente usate in un'applicazione**) è piccolo ($n \ll |U|$), si ha uno spreco di memoria.

Possibile strategia alternativa

- si usa una tabella di dimensioni $m \approx n$ (anche se $m \ll |U|$).
- si usa una funzione che trasforma la chiave per trovare una posizione di memorizzazione nella tabella: $h: U \rightarrow \{0, \dots, m-1\}$.
- h è una **funzione hash** (funzione che **non** può essere iniettiva!).
- $h(k)$ è il **valore hash** della chiave k .

Tavole hash

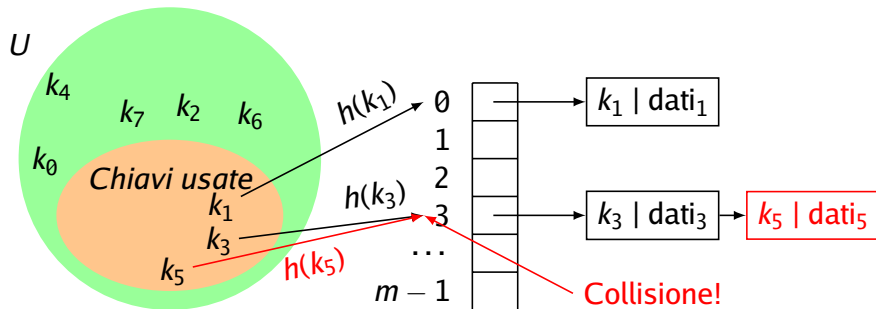
Idea generale



- $m \ll |U|$
- Funzione hash $h: U \rightarrow \{0, \dots, m-1\}$
 - disperdere in modo uniforme le chiavi;
 - non può essere iniettiva su tutto U quando $m \ll |U|$, quindi le collisioni sono possibili.
 - se sulle sole chiavi usate non ci sono collisioni, allora è una funzione **hash perfetta** per quell'insieme di chiavi.

Tavole hash

Idea generale



- $m \ll |U|$
- Funzione hash $h: U \rightarrow \{0, \dots, m-1\}$
 - disperdere in modo uniforme le chiavi;
 - non può essere iniettiva su tutto U quando $m \ll |U|$, quindi le collisioni sono possibili.
 - se sulle sole chiavi usate non ci sono collisioni, allora è una funzione **hash perfetta** per quell'insieme di chiavi.

Tabella hash

Funzione hash: come scegliere?

- Una funzione hash h ideale dovrebbe determinare un valore $h(k)$ in modo casuale e indipendente nell'insieme $\{0, 1, \dots, m - 1\}$.
- Il valore $h(k)$ deve poi essere sempre lo stesso ogni volta che si invoca $h(k)$.
- Una funzione h di questo tipo è detta **funzione di hash uniforme e indipendente** od *oracolo casuale*.
- Tali funzioni ideali sono solo teoriche perché la casualità uniforme non è realizzabile praticamente.
- Considereremo delle funzioni hash che approssimano il concetto di “funzione di hash uniforme e indipendente”.

Tabella hash

Funzione hash: come scegliere?

- Una *buona* funzione hash soddisfa (in modo approssimato) l'ipotesi di hashing uniforme e indipendente.
- In altre parole, dovrebbe garantire che ogni chiave ha la stessa probabilità di essere assegnata a una qualunque delle m celle.
- In generale la distribuzione di probabilità delle chiavi non è nota.
- Una buona funzione dovrebbe quindi determinare valori che ci si aspetta siano indipendenti da qualunque distribuzione possa esistere nelle chiavi.
- **Hashing statico** prevede di usare una sola funzione hash che funzioni bene su tutti i dati. È una tecnica efficiente ma non può garantire un certo comportamento medio se non si fanno assunzioni sulla distribuzione delle chiavi.
- **Hashing randomizzato** prevede di scegliere una funzione hash da usare in modo casuale da una famiglia. In questo modo è possibile garantire un certo comportamento medio indipendentemente dalla distribuzione delle chiavi.

Tabella hash

Funzione hash: conversione verso gli interi

- Si considerano per prime funzioni hash **statiche** che operano con chiavi date da valori interi.
- Se l'universo U non è d'interi, si deve prima convertire una chiave in un intero e poi applicare la funzione hash.

Tabella hash

Funzione hash: conversione verso gli interi

Conversione chiave alfanumerica:

- sia $k = c_n c_{n-1} \cdots c_0$ dove c_i è un char;
- se si vuole determinare un intero in corrispondenza biunivoca con k , allora la conversione è $k' = \sum (c_i \cdot 65536^i)$ dove la base è 65536 perché in Java un char è di 16 bit*.
- se si ha la certezza che i caratteri sono del sottoinsieme Latin1 (set ASCII più alcune lettere accentate di vario tipo e segni grafici), allora la base può essere 256;
- se si fanno ulteriori restrizioni sul set di caratteri possibili e se si riassegnano gli ordinali ai caratteri possibili a partire da 0, allora la base può essere ulteriormente diminuita in valore.

*Per i caratteri Unicode che richiedono più coppie di 16 bit per essere rappresentati, si considerano solo i primi 16 bit

Tabella hash

Funzione hash: conversione verso gli interi

Nota!

- Le prime due conversioni proposte determinano un valore intero che supera il limite rappresentabile con un `int` Java già per stringhe di 2 caratteri;
- non si avranno comunque errori di runtime perché la somma finale verrà troncata per essere rappresentata da un `int`;
- cosa si perde allora?
- Perché si possono comunque adottare tali codifiche?

Tabella hash

Funzione hash: conversione verso gli interi (overflow)

- Quando una codifica produce un valore troppo grande per un `int`, in Java il risultato viene mantenuto nei soli $w = 32$ bit (rappresentazione in complemento a 2).
- Dal punto di vista matematico, questo equivale a calcolare la codifica modulo 2^w :

$$\text{int}(k') = k' \bmod 2^{32}.$$

- Quindi la conversione non è più biunivoca: stringhe diverse possono dare lo stesso `int` (collisioni).
- Questo non è un problema per l'hashing: anche la funzione hash finale $h: U \rightarrow \{0, \dots, m-1\}$ non può essere iniettiva. Qui la conversione serve soprattutto a mescolare i caratteri in un numero intero prima di applicare h .

Mini-esempio (idea)

Se si usa la base 256 (caratteri Latin1), una stringa $k = c_1 c_0$ si codifica come $k' = c_0 + 256 c_1$. Se la stringa è lunga, la somma continua ma il risultato viene poi considerato modulo 2^{32} : è una forma di *polynomial rolling hash* con modulo 2^{32} .

Tabella hash

Hashing statico

Funzione hash $h: U \rightarrow \{0, \dots, m-1\}$

- Esistono diversi metodi che producono funzioni di buona qualità:
 - metodo della **divisione**.
 - metodo della **moltiplicazione**.
- tutte hanno dominio l'insieme degli interi.

Tabella hash

Hashing statico: metodo divisione

- Sia m la dimensione della tabella;
- dato che $h()$ deve restituire sempre un indice valido nell'intervallo $[0, \dots, m - 1]$, la più semplice funzione hash è

$$h(k) = k \bmod m$$

- Richiede solo 1 operazione.
- Scelta di m :
 - m è critico: meglio scegliere numero primo non vicino a potenze di 2;
 - oppure, un buon m è un numero con fattori primi > 20 (Lum, 1971);
- Non c'è garanzia che anche con un buon m il metodo dia buone prestazioni nel caso medio.

*L'operatore modulo 'mod' è realizzato dall'operatore '%' in Java.

Tabella hash

Hashing statico: metodo divisione

Esempi negativi di $h()$ basati sul metodo della divisione

- se $m = 2^p$, allora $h(k)$ = i p bit meno significativi di k ;
- Per esempio, se $m = 2^{12} = 4096$, i seguenti valori
 $h(40960) = h(45056) = h(49152) = h(53248) = 0$ perché hanno i 12 bit meno significativi pari a 0.

Tabella hash

Hashing statico: metodo moltiplicazione

- Sia m la dimensione della tabella e $0 < A < 1$ una costante.
- La chiave k è moltiplicata per una costante A e si considera la parte frazionaria.
- Quindi si moltiplica tale parte per m per avere il valore hash:

$$h(k) = \lfloor m((k \cdot A) \bmod 1) \rfloor$$

dove $(k \cdot A) \bmod 1$ è la parte frazionaria di $(k \cdot A)$.

- Note:
 - $k \cdot A$ è un valore decimale $< k$.
 - non essendo $k \cdot A$ un `int`, $k \cdot A \bmod 1$ è la parte decimale del valore $k \cdot A$, quindi è < 1 .
 - m **non** è critico. Valore tipico $m = 2^p$, con $p \in \mathbb{N}$.
 - Donald Knuth suggerisce $A \approx \frac{\sqrt{5}-1}{2} \approx 0.6180$ (sezione aurea).

Tabella hash

Hashing statico: metodo moltiplica e scorri

- Si consideri il metodo della moltiplicazione fissando $m = 2^l$ dove l è un intero $0 < l \leq w$ e w è # bit della parola macchina.
- Si scelga poi $a = A \cdot 2^w$, con A del metodo della moltiplicazione $\Rightarrow 0 < a < 2^w$.
- Chiave k è rappresentabile in w bit.
- Il valore $k \cdot a = r_1 2^w + r_0$ è un intero di $2w$ bit.
- Il metodo **moltiplica e scorri** prevede di considerare come hash gli l bit più significativi di r_0 .

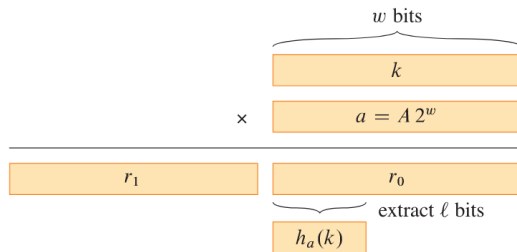


Tabella hash

Hashing statico: metodo moltiplica e scorri

- Il valore $k \cdot a = r_1 2^w + r_0$ è un intero di $2w$ bit.
- Per ottenere gli l bit più significativi di r_0 si dovrebbe dividere r_0 con 2^{w-l} e prendere la parte intera inferiore.
- Tale divisione si può fare con uno scorrimento logico a destra (right logic shift) $>>>$:

$$h_a(k) = ((k \cdot a) \bmod 2^w) >>> (w - l)$$

- L'operazione $\bmod 2^w$ è implicita dato che la parola macchina è di w bit.
- **Vantaggio:** fissati w e l , h_a si può calcolare con una moltiplicazione e uno scorrimento logico a destra.

*In Java, l'operatore scorrimento logico a destra è rappresentato come ' $>>>$ '.

Tabella hash

Hashing statico: metodo moltiplica e scorri

Esempio 2

- Assumiamo che la memoria abbia celle di dimensione $w = 8$ bit.
- Vogliamo usare una tabella di $m = 16$ celle.
 $m = 2^4 = 16 \Rightarrow l = 4$.
- Sia $k = 202$ (in binario $k = 11001010_2$) e $a = 173$ (in binario $a = 10101101_2$).
- Calcolo:
$$k \cdot a = 202 \cdot 173 = 34946,$$
$$r_0 = (k \cdot a) \bmod 2^w = 34946 \bmod 256 = 130 = 10000010_2,$$
$$h_a(k) = r_0 \ggg (w - l) = 10000010_2 \ggg 4 = 1000_2 = 8.$$
- Quindi la chiave finisce nella cella $8 \in \{0, \dots, 15\}$.
- Nota: $\ggg$ inserisce zeri a sinistra (a differenza di $\gg$ che mantiene il bit di segno).

Tabella hash

Hashing randomizzato: idea generale

- Se si usa una funzione hash statica, se si devono inserire n chiavi, nel caso peggiore, tutte vengono associate alla stessa posizione. Quindi $n - 1$ collisioni.
- Unico modo efficace per migliorare la situazione è scegliere *casualmente* la funzione hash prima di ogni esecuzione, in modo che il caso peggiore non possa verificarsi ogni volta che si presentano le n chiavi del punto precedente $\Rightarrow$ hashing randomizzato .
- Si dimostra che, per il caso speciale di hashing randomizzato chiamato hashing universale , il comportamento medio è *buono* e indipendente dalla distribuzione delle chiavi in input.

Tabella hash

Hashing randomizzato: idea generale

- Nell'hashing randomizzato, all'inizio di ogni esecuzione, si deve scegliere la funzione hash in modo casuale da una famiglia opportuna.
- Ogni esecuzione potrà mappare quindi le stesse n chiavi in posizioni diverse.
- Nessun input quindi potrà rappresentare sistematicamente il caso peggiore.

Tabella hash

Hashing randomizzato: hashing universale

Definizione 1 (Hashing universale)

- Sia $\mathcal{H}$ una famiglia finita di funzioni hash statiche aventi dominio U e codominio l'intervallo $\{0, 1, \dots, m-1\}$.
- Una famiglia $\mathcal{H}$ è detta **universale** se, per ogni coppia di chiavi diverse $k_i, k_j \in U$, il **numero** di funzioni hash $h \in \mathcal{H}$ che fanno **collidere** le due chiavi (cioè per cui $h(k_i) = h(k_j)$) è al più $|\mathcal{H}|/m$.
- Fissate due chiavi diverse k_i, k_j , definiamo l'insieme delle funzioni che le fanno collidere: $C_{ij} = \{h \in \mathcal{H} \mid h(k_i) = h(k_j)\}$.
- La definizione dice $|C_{ij}| \leq |\mathcal{H}|/m$. Se scegliamo h uniformemente a caso da $\mathcal{H}$, allora

$$\Pr\{h(k_i) = h(k_j)\} = \Pr\{h \in C_{ij}\} = \frac{|C_{ij}|}{|\mathcal{H}|} \leq \frac{1}{m}.$$

Tabella hash

Hashing randomizzato: classificazione

Sia $\mathcal{H}$ una famiglia finita di funzioni hash statiche aventi dominio U e codominio l'intervallo $\{0, 1, \dots, m-1\}$ e sia h una qualsiasi funzione hash in $\mathcal{H}$ scelta in modo casuale.

- $\mathcal{H}$ è **uniforme** se $\forall k \in U$ e per ogni cella $q \in \{0, 1, \dots, m-1\}$, $\Pr\{h(k) = q\} = 1/m$.
- $\mathcal{H}$ è **ε -universale** se $\forall k_i, k_j \in U, k_i \neq k_j$, $\Pr\{h(k_i) = h(k_j)\} \leq \varepsilon$.
- $\mathcal{H}$ è **universale** se è ε -universale con $\varepsilon = 1/m$, cioè se $\forall k_i \neq k_j$, $\Pr\{h(k_i) = h(k_j)\} \leq 1/m$.

Tabella hash

Hashing randomizzato: Hashing uniforme e indipendente

Definizione 2 (Hashing uniforme e indipendente)

Fissato un insieme S di n chiavi (le chiavi dell'istanza), si sceglie una funzione hash $h: S \rightarrow \{0, \dots, m-1\}$ uniformemente a caso tra **tutte** le m^n funzioni possibili.

- **(Uniformità)** $\forall k \in S$ e $\forall q \in \{0, \dots, m-1\}$, $\Pr\{h(k) = q\} = 1/m$.
- **(Indipendenza)** per due chiavi distinte $k_1 \neq k_2$ e due celle q_1, q_2 , $\Pr\{h(k_1) = q_1 \wedge h(k_2) = q_2\} = 1/m^2$ (e analogamente per più chiavi).
- Quindi $\Pr\{h(k_i) = h(k_j)\} = \sum_{q=0}^{m-1} \Pr\{h(k_i) = q \wedge h(k_j) = q\} = m \cdot \frac{1}{m^2} = \frac{1}{m}$: una famiglia uniforme e indipendente è anche universale.
- Il viceversa non vale in generale: esistono famiglie universali che non sono uniformi e indipendenti.

Tabella hash

Hashing randomizzato: Hashing uniforme e indipendente

Esempio 3 ($\mathcal{H} = \{I\}$ è universale ma non uniforme/indipendente)

- Si assuma $U = \{0, \dots, m-1\}$ e $I: U \rightarrow U$ funzione identità.
- **Universalità:** scegliendo h a caso da $\mathcal{H}$ si ottiene sempre $h = I$. Per ogni $a \neq b$ vale $I(a) = a \neq b = I(b)$, quindi

$$\Pr\{h(a) = h(b)\} = 0 \leq 1/m.$$

- **Non uniformità:** fissato k , $\Pr\{h(k) = k\} = 1$ e per ogni $q \neq k$, $\Pr\{h(k) = q\} = 0 \neq 1/m$.
- **Non indipendenza:** per due chiavi distinte $k_1 \neq k_2$, la coppia $(h(k_1), h(k_2))$ è deterministica (k_1, k_2) , quindi ad esempio

$$\Pr\{h(k_1) = k_1 \wedge h(k_2) = k_2\} = 1 \neq 1/m^2.$$

Tabella hash

Hashing randomizzato: progettare una famiglia di hashing universale

Famiglie universali

Sono il tipo più semplice di famiglie di funzioni hash per le quali si può dimostrare l'efficienza per ogni insieme di dati in input.

- Due modi comuni per costruire una famiglia (di hashing) universale:
 - 1 usando la teoria dei numeri.
 - 2 usando una variante randomizzata del metodo moltiplica e scorri.
- Si presentano i due metodi senza dimostrazioni della loro correttezza per dare un'idea della fattibilità delle famiglie di hashing universale.

Tabella hash

Hashing randomizzato: famiglie di hashing universale basate sulla teoria dei numeri

- Sia p un numero primo tale che ogni chiave $k \in U$ possa essere associata a un intero nell'intervallo $[0, \dots, p-1]$.
- Si assume poi che $p > m$, dimensione tabella.
- Sia $\mathbb{Z}_p = \{0, \dots, p-1\}$ e $\mathbb{Z}_p^* = \{1, \dots, p-1\}$.
- Scelti $a \in \mathbb{Z}_p^*$ e $b \in \mathbb{Z}_p$, si definisce funzione hash

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m$$

- Dati p, m , la famiglia di hashing universale $\mathcal{H}_{pm}$ è formata da tutte le funzioni hash del tipo h_{ab} :

$$\mathcal{H}_{pm} = \{h_{ab}() \mid a \in \mathbb{Z}_p^*, b \in \mathbb{Z}_p\}$$

- m può essere qualsiasi. $|\mathcal{H}_{pm}| = p(p-1)$.

Tabella hash

Hashing randomizzato: famiglie di hashing $2/m$ -universale basate sul metodo moltiplica e scorri

- Ricordiamo che una funzione hash moltiplica e scorri è definita come $h_a(k) = ((k \cdot a) \bmod 2^w) \ggg (w - l)$, dove $a = A \cdot 2^w$, con A del metodo della moltiplicazione, w è il numero di bit usati per rappresentare gli interi ed $m = 2^l$.
- Si dimostra che la famiglia di hashing $2/m$ -universale $\mathcal{H}$ è formata da tutte le funzioni hash del tipo h_a con a dispari:

$$\mathcal{H} = \{h_a() \mid 1 \leq a < 2^w, a \text{ dispari}\}$$

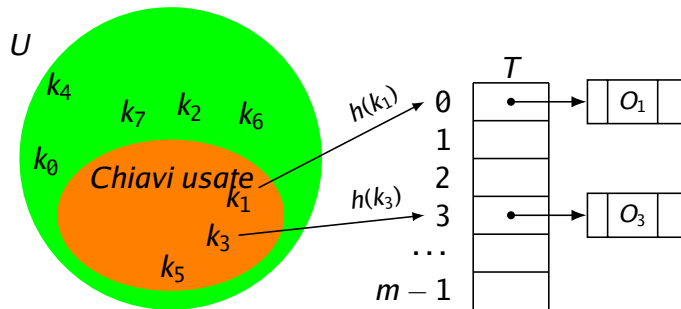
- m deve quindi essere una potenza di 2. $|\mathcal{H}| = 2^{w-1}$.
- La velocità con cui si può calcolare $h_a(k)$ compensa il fatto che la probabilità di collisione sia $2/m$ anziché $1/m$.

* Nella versione italiana del [CLRS] ci sono due errori nella definizione della famiglia.

- Anche con funzioni hash da famiglie di hashing universale, può accadere che più di una chiave può essere assegnata alla stessa posizione (**collisione**).
- Esistono diverse tecniche per gestire le collisioni:
 - concatenamento (chaining).
 - indirizzamento aperto (open addressing):
 - reindirizzamento lineare (linear probing).
 - doppio hashing.

Tabella hash

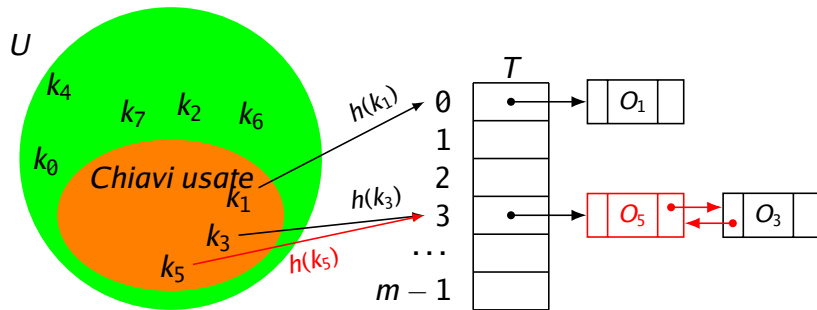
Collisioni: risoluzione mediante concatenamento



- Tutti gli elementi con medesima chiave hash sono posti su una lista;
- $\text{INSERT}(o)$: inserisci o in testa nella lista $T[h(o.\text{key})]$;
- $\text{SEARCH}(k)$: restituisci l'oggetto con chiave k nella lista $T[h(k)]$;
- $\text{DELETE}(o)$: cancella l'oggetto o nella lista $T[h(o.\text{key})]$.

Tabella hash

Collisioni: risoluzione mediante concatenamento



- Tutti gli elementi con medesima chiave hash sono posti su una lista;
- $\text{INSERT}(o)$: inserisci o in testa nella lista $T[h(o.\text{key})]$;
- $\text{SEARCH}(k)$: restituisci l'oggetto con chiave k nella lista $T[h(k)]$;
- $\text{DELETE}(o)$: cancella l'oggetto o nella lista $T[h(o.\text{key})]$.

Tabella hash

Collisioni: risoluzione mediante concatenamento

Prestazioni caso peggiore

Operazione Caso peggiore	
--------------------------	--

inserisci	$\Theta(1)$
ricerca	$\Theta(n')$
cancella	$\Theta(1)$

- n' è la lunghezza della lista considerata, n numero elementi totale da memorizzare. Nel caso peggiore, $n' = n$.
- In INSERT si assume che l'elemento non sia presente.
- In DELETE(o), o è un elemento della tabella: contiene anche i puntatori *prev* e *next*, quindi la cancellazione richiede tempo costante. **Attenzione! Questa versione di DELETE(o) è quella considerata in [CLRS]. Più avanti vedremo una versione che accetta in input la chiave!**

Tabella hash

Collisioni: risoluzione mediante concatenamento

Prestazioni caso medio

- m lunghezza tabella,
- n_j lunghezza lista $T[j]$,
- $n = n_0 + \dots + n_{m-1}$ numero totale elementi.
- $\alpha = \frac{n}{m}$ è **indice/fattore di carico**.
- Nelle tabelle hash con concatenamento è accettabile avere $\alpha > 1$ normalmente, anche se è preferibile avere $\alpha \approx 1$.
- **Assunzione hashing uniforme e indipendente:** qualsiasi chiave ha la stessa probabilità di essere mandata in una qualsiasi delle m celle della tabella, indipendentemente dal trattamento delle altre chiavi $\Rightarrow E[n_j] = \alpha$.
- Costo di calcolo valore hash $\Theta(1)$.

Tabella hash

Collisioni: risoluzione mediante concatenamento

Teorema 1

*In una tabella hash con concatenamento, una ricerca **senza successo** richiede in media tempo $\Theta(1 + \alpha)$ nell'ipotesi hashing uniforme e indipendente.*

Dimostrazione.

Ogni lista delle m celle ha una lunghezza media α . Quindi la ricerca di una chiave non presente richiede $\Theta(1)$ per calcolare quale lista scorrere e $\Theta(\alpha)$ per scorrere la lista. □

Tabella hash

Collisioni: risoluzione mediante concatenamento

Prestazioni caso medio per la ricerca con successo

- Si determina l'ordine di grandezza del tempo medio di esecuzione in funzione di quanti confronti si devono fare per trovare l'elemento più il tempo per calcolare il valore hash della chiave cercata.
- Nel caso in cui si cerca una chiave presente, il calcolo del tempo atteso è meno diretto perché *si deve tener conto che non è necessario scorrere tutta una lista per trovare l'elemento*.
- Per calcolare correttamente il tempo del caso medio, si devono considerare:
 - il numero di elementi n presenti nella tabella;
 - il numero medio di elementi che sono presenti in una lista prima dell'elemento cercato;
 - calcolare quindi il valore medio rispetto a n di 1 (costo di confronto dell'elemento cercato) + numero medio di elementi prima dell'elemento cercato;

* L'analisi fatta in CLRS edizione III è errata. Di seguito si riporta l'analisi corretta.

Tabella hash

Collisioni: risoluzione mediante concatenamento

Prestazioni caso medio per la ricerca con successo (continua)

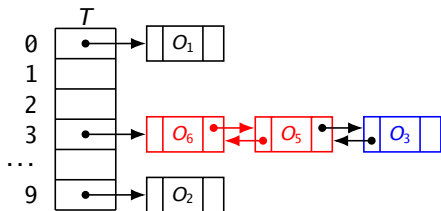
- Si assume che l'elemento da cercare abbia la stessa probabilità di essere uno qualunque degli n elementi presenti in tabella.
- Numero di elementi esaminati nella ricerca con successo di x
 $= 1 + \text{numero elementi prima di } x \text{ nella lista}$
- Un elemento y è prima di x nella lista se ha lo stesso valore hash ed è stato aggiunto nella tabella **dopo** x , perché si inserisce in testa alla lista.

Tabella hash

Collisioni: risoluzione mediante concatenamento

Prestazioni caso medio per la ricerca con successo (continua)

- x_i denota l' i -esimo elemento inserito nella hash table.
- Esempio: se inserisco, in ordine, gli oggetti O_1, O_3, O_5, O_2, O_6 , allora $x_1 = O_1, x_2 = O_3, x_3 = O_5, x_4 = O_2$ e $x_5 = O_6$.
- Nelle slide seguenti si assume che
 $h(O_1) = 0, h(O_3) = h(O_5) = h(O_6) = 3, h(O_2) = 9 = m - 1$.



- $x_1 = O_1, x_2 = O_3, x_3 = O_5, x_4 = O_2, x_5 = O_6$
- Si assume di voler cercare x_2

Tabella hash

Collisioni: risoluzione mediante concatenamento

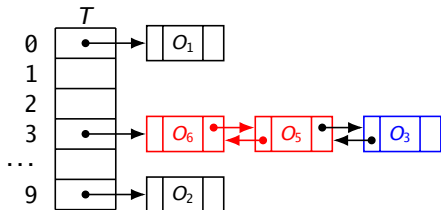
Prestazioni caso medio per la ricerca con successo (continua)

- Per ciascun slot q della tabella, e per ciascuna coppia di chiavi distinte k_i e k_j , sia

$$X_{ijq} = I\{\text{si cerca } x_i, h(k_i) = q, h(k_j) = q\}$$

(v.c. indicatrice che x_i e x_j sono nella stessa lista q).

- Nell'ipotesi di hashing uniforme e indipendente (e assumendo che l'elemento cercato sia scelto uniformemente tra gli n presenti): $\Pr\{\text{si cerca } x_i\} = 1/n$, $\Pr\{h(k_i) = q\} = 1/m$ e $\Pr\{h(k_j) = q\} = 1/m$.
- Quindi, $\Pr\{X_{ijq} = 1\} = \frac{1}{nm^2} \Rightarrow E[X_{ijq}] = \frac{1}{nm^2}$.



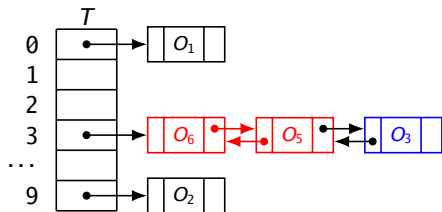
- $x_1 = O_1, x_2 = O_3, x_3 = O_5, x_4 = O_2, x_5 = O_6$
- Si assume di voler cercare x_2
- Ci sono $5 \cdot 4 \cdot 10 = 200$ variabili X_{ijq} .
- $X_{ijq} = 0$ per $q \neq 3$ o $i \neq 2$ o $j = [1, 4]$
- $X_{2,3,3} = 1, X_{2,5,3} = 1$

Tabella hash

Collisioni: risoluzione mediante concatenamento

Prestazioni caso medio per la ricerca con successo (continua)

- Per ciascun x_j , sia $Y_j = I\{x_j \text{ è prima dell'elemento cercato nella lista}\}$.
- $Y_j = 1$ quando x_j è nella stessa lista dell'elemento cercato e appare **più vicino alla testa** della lista.
(qui: INSERT inserisce in testa, quindi “più vicino alla testa” $\Leftrightarrow$ inserito dopo).
- Vale $Y_j = \sum_{q=0}^{m-1} \sum_{i=1}^{j-1} X_{ijq}$



- $x_1 = O_1, x_2 = O_3, x_3 = O_5, x_4 = O_2, x_5 = O_6$
- Si assume di voler cercare x_2
- $X_{2,3,3} = 1, X_{2,5,3} = 1$
- $Y_1 = 0, Y_2 = n.d., Y_3 = 1, Y_4 = 0, Y_5 = 1$

Tabella hash

Collisioni: risoluzione mediante concatenamento

Prestazioni caso medio (continua)

- Sia $Z = \sum_{j=1}^n Y_j$, v.c.i. che conta quanti sono gli elementi prima dell'elemento cercato X_j .
- La media cercata è quindi $E[1 + Z]$

$$\begin{aligned} E[1 + Z] &= 1 + E \left[\sum_{j=1}^n \sum_{q=0}^{m-1} \sum_{i=1}^{j-1} X_{ijq} \right] \\ &= 1 + \sum_{j=1}^n \sum_{q=0}^{m-1} \sum_{i=1}^{j-1} \frac{1}{nm^2} \\ &= 1 + m \frac{n(n-1)}{2} \frac{1}{nm^2} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \end{aligned}$$

- Aggiungendo il costo $\Theta(1)$ per calcolare il valore hash, in media il costo di ricerca con successo diventa $\Theta(2 + \frac{\alpha}{2} - \frac{\alpha}{2n}) = \Theta(1 + \alpha)$.

Tabella hash

Collisioni: risoluzione mediante concatenamento

Prestazioni caso medio

Operazione	Caso peggiore	Caso medio
inserisci	$\Theta(1)$	$\Theta(1)$
ricerca	$\Theta(n)$	$\Theta(\alpha + 1)$
cancella	$\Theta(1)$	$\Theta(1)$

- $\alpha = \frac{n}{m}$ è indice di carico, dove m è dimensione tabella;
- nel caso in cui $n = O(m)$ (numero di elementi è proporzionale al numero delle celle disponibili), nel caso medio tutte le operazioni costano $O(\frac{cm}{m} + 1) = O(1)$.

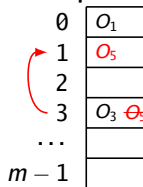
Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto

- Tutti gli elementi sono memorizzati nella tabella stessa.
- In caso di conflitto, si usa un'altra cella della tabella per memorizzare il nuovo oggetto:

$$h(k_1) = 0$$

$$h(k_3) = h(k_5) = 3$$



0	O_1
1	O_5
2	
3	O_3 O_5
...	
$m - 1$	

- La ricerca di una cella nuova libera in caso di conflitto si chiama **ispezione**.
- Teoricamente si possono considerare le altre $m - 1$ celle.
- Si estende perciò la funzione hash come
$$h: U \times \{0, 1, \dots, m - 1\} \rightarrow \{0, 1, \dots, m - 1\}$$
- Da qui in poi la funzione hash non estesa è denotata come $h'()$.

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto

- Le procedure d'INSERT, SEARCH e DELETE devono essere adeguate perché non tutte le celle della tabella hash contengono un oggetto con chiave il cui hash è l'indirizzo della cella stessa.

Algoritmo: INSERT(T , O)

Input: tabella hash T ; oggetto da inserire o **che non è presente**.

Result: o inserito se la tabella ha spazio, errore "Overflow" altrimenti.

```
1:  $i = 0$ 
2: do
3:    $j = h(o.key, i)$ 
4:   if  $T[j] == NULL \vee T[j] == DELETE$  then                                // DELETE spiegato dopo
5:      $T[j] = o$ 
6:     return "Inserito"
7:    $i++$ 
8: while  $i \neq m$ 
9: return "Overflow"
```

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto

Algoritmo: SEARCH(T, k)

Input: tabella hash T ; chiave da cercare k .

Output: indice posizione di k in T se presente, NULL altrimenti.

```
1:  $i = 0$ 
2: do
3:    $j = h(k, i)$ 
4:   if  $T[j] == \text{NULL}$  then return NULL
5:   if  $T[j] \neq \text{DELETE} \wedge T[j].\text{key} == k$  then return } j
6:    $i = i + 1$ 
7: while  $i \neq m$ 
8: return NULL
```

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto

Algoritmo: DELETE(T, I)

Input: tabella hash T , indice cella da cancellare i .

Result: marca “DELETE” la cella i .

// Usare SEARCH per cercare l'indice i associato a una chiave.

1: $T[i] = DELETE$

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto (perché serve DELETE)

- In SEARCH, incontrare una cella NULL significa: la sequenza d'ispezione si ferma (la chiave non può essere più avanti).
- Se in DELETE mettessimo NULL, potremmo “tagliare” la sequenza e rendere SEARCH sbagliata (falso negativo).
- Per questo si usa un marcatore DELETE (*tombstone*):
 - SEARCH non si ferma su DELETE e continua a ispezionare;
 - INSERT può riusare una cella DELETE.
- Effetto collaterale: molte celle DELETE aumentano le ispezioni (la tabella “sembra” più piena).
- Soluzione pratica: quando i DELETE diventano tanti, si esegue un rehash (si ricostruisce la tabella reinserendo solo gli elementi presenti).

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto (ipotesi dell'analisi)

- m = numero di celle, n = numero di chiavi presenti, $\alpha = \frac{n}{m}$ (con indirizzamento aperto vale $\alpha < 1$).
- Le formule sul numero atteso di **ispezioni** valgono sotto l'ipotesi di hashing con permutazioni indipendenti e uniformi :
 - per ogni chiave k , la sequenza $\langle h(k, 0), \dots, h(k, m-1) \rangle$ è un **ordine casuale** (permutazione uniforme) di $\{0, \dots, m-1\}$, quindi ogni cella viene ispezionata una sola volta;
 - le sequenze (per chiavi diverse) sono indipendenti.
- **Nota:** è un'ipotesi **più forte e diversa** da “hashing uniforme e indipendente”: qui si assume casuale l'intera sequenza di ispezioni.
- Inoltre, nell'analisi classica di [CLRS] si assume **assenza di cancellazioni** : senza questa ipotesi i marcatori DELETE possono degradare le prestazioni e le formule non descrivono più bene il comportamento.
- In pratica, con molte cancellazioni si usa periodicamente un **rehash** per eliminare i DELETE.

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto

Operazione	Caso medio
ricerca con successo	$\frac{1}{\alpha} \ln \left(\frac{1}{1-\alpha} \right)$
ricerca con insuccesso	$\frac{1}{1-\alpha}$

- In una tabella riempita per la metà ($\alpha = 1/2$)
 - ricerca di una chiave **inesistente** (e quindi un inserimento) richiede mediamente $1/(1 - 1/2) = 2$ ispezioni.
 - ricerca di una chiave **esistente** richiede mediamente $2 \ln \left(\frac{1}{1-1/2} \right) = 2 \ln 2 \approx 1.387$ ispezioni.
- In una tabella riempita al 75% ($\alpha = .75$)
 - ricerca di una chiave **inesistente** richiede mediamente $1/(1 - .75) = 4$ ispezioni.
 - ricerca di una chiave **esistente** richiede mediamente $4/3 \ln(4) \approx 1.848$ ispezioni.

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto

- Ci sono 3 tecniche per calcolare le sequenze d'ispezione:
 - ispezione lineare;
 - ispezione quadratica (vedere [CLRS]);
 - doppio hashing;

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto con ispezione lineare

- In inserimento, in caso di conflitto, si ricerca la prima cella libera, scandendo le celle a una a una, a partire da quella adiacente a quella corretta:
- Si supponga che si inserisca, in ordine, k_1 , k_3 , k_5 e k_6 e che $h'(k_1) = 0$, $h'(k_3) = h'(k_5) = h'(k_6) = 3$;
- L'inserimento di k_5 richiede la ricerca di 1 cella libera: sufficiente 1 salto;
- L'inserimento di k_6 richiede la ricerca di 1 cella libera: necessari 2 salti;

0	O_1
1	
2	
3	O_3
...	
$m - 1$	

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto con ispezione lineare

- In inserimento, in caso di conflitto, si ricerca la prima cella libera, scandendo le celle a una a una, a partire da quella adiacente a quella corretta:
- Si supponga che si inserisca, in ordine, k_1 , k_3 , k_5 e k_6 e che $h'(k_1) = 0$, $h'(k_3) = h'(k_5) = h'(k_6) = 3$;
- L'inserimento di k_5 richiede la ricerca di 1 cella libera: sufficiente 1 salto;
- L'inserimento di k_6 richiede la ricerca di 1 cella libera: necessari 2 salti;

0	O_1	
1		
2	O_5	
3	O_3	Θ_5
...		
$m - 1$		

1 ↗

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto con ispezione lineare

- In inserimento, in caso di conflitto, si ricerca la prima cella libera, scandendo le celle a una a una, a partire da quella adiacente a quella corretta:
- Si supponga che si inserisca, in ordine, k_1 , k_3 , k_5 e k_6 e che $h'(k_1) = 0$, $h'(k_3) = h'(k_5) = h'(k_6) = 3$;
- L'inserimento di k_5 richiede la ricerca di 1 cella libera: sufficiente 1 salto;
- L'inserimento di k_6 richiede la ricerca di 1 cella libera: necessari 2 salti;

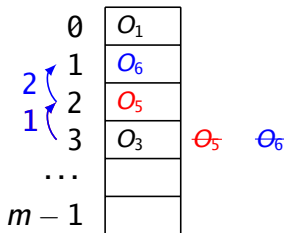


Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto con ispezione lineare

- La funzione hash che realizza ispezione lineare è

$$h(k, i) = (h'(k) + i) \% m, i = [0, \dots, m - 1]$$

Nota!

Il metodo soffre del cosiddetto *Problema del clustering primario*:

- cluster: sequenza di celle adiacenti occupate;
- un cluster tende a crescere velocemente, vanificando la funzione hash.
- Non soddisfa l'ipotesi di hashing con permutazioni indipendenti e uniformi perché può generare al massimo m permutazioni differenti invece di $m!$
- Questa tecnica è comunque interessante perché si adatta bene a sistemi che hanno la memoria organizzata in modo gerarchico (cache/RAM/disco).

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto con doppio hashing

- In inserimento, in caso di conflitto, si ricerca una cella libera mediante l'uso di un'altra funzione hash.
- La funzione hash che realizza ispezione con doppio hashing è

$$h(k, i) = (h'(k) + i \cdot p(k)) \bmod m$$

- $i = [0, \dots, m - 1]$.
- $p(k)$ è un'altra funzione hash (funzione probe).
- $p(k)$ deve garantire di poter accedere a tutte le celle in m chiamate.

Tabella hash

Collisioni: risoluzione mediante indirizzamento aperto con doppio hashing

Nota!

- problema clustering primario non presente;
- $p()$ deve garantire la copertura di tutto lo spazio array
 - condizione sufficiente è che $\text{mcd}(p(k), m) = 1$;
Esempio: m primo e $0 < p(k) < m$:
 $\Rightarrow m = 701; p(k) = 1 + (k \bmod 500)$.
 - altra condizione sufficiente è che se $m = 2^c$, allora $0 < p(k) < m$ sia dispari.
Esempio: $m = 512; p(k) = 1 + 2(k \bmod m/2)$.
- Il doppio hashing è in generale migliore delle ispezioni lineari o quadratiche.
- Anche se non soddisfa l'ipotesi di hashing con permutazioni indipendenti, le permutazioni prodotte hanno molte caratteristiche delle permutazioni casuali.

- Studiare il capitolo 11 fino all'11.5 escluso [CLRS]. Non è necessario conoscere il dettaglio delle dimostrazioni 11.4, 11.6, e 11.7.
- Risolvere gli esercizi 11.1-1, 11.2-1, 11.2-2, 11.2-3, 11.3-1, 11.3-4, 11.4-3 [CLRS].